

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — Architecture Numérique à Trois Niveaux 8

1.1 Le modèle trois niveaux 9

1.2 Le niveau applicatif : serveur, répartition et interpréteur 10

1.3 Le niveau de données et le référentiel applicatif 12

1.4 L'intégration des modules fonctionnels 13

1.5 Transactions, verrous et performances 15

1.6 Évolution, migration et architectures de remplacement 19

Annexes 30

A.1 Liste des abréviations 31

A.2 Table de notation 31

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Le patron de conception qui underlie tous les systèmes d'entreprise

Partie 1

Architecture Numérique à Trois Niveaux

Le modèle trois niveaux sépare trois responsabilités qui ont toujours été ICI confondues : ce que l'utilisateur voit, ce que le logiciel décide, et ce qui est stocké. Cette séparation, formalisée dans les années 1970, reste la grille de lecture de tous les systèmes d'information d'entreprise, du progiciel intégré des années 1990 aux platesformes en mémoire et aux services distribués d'aujourd'hui. On l'étudie ici à travers le cas de référence le plus répandu au monde, puis on en tire les règles qui permettent de décider où couper.

Objectifs de la partie

- Nommer les trois niveaux, leurs composants et le protocole qui les relie.
- Expliquer le rôle du serveur de répartition et du serveur de messages dans une application d'entreprise.
- Distinguer le référentiel d'une base de données, et comprendre pourquoi il conditionné toute évolution.
- Suivre le parcours complet d'une transaction, du poste de travail à l'écriture en base.
- Justifier un découpage de charge entre plusieurs serveurs sans rompre la cohérence.
- Évaluer une migration vers une plateforme en mémoire : ce qu'elle gagne, ce qu'elle coûte.

Plan de la partie

1. Le modèle trois niveaux
2. Le niveau applicatif : serveur, répartition et interpréteur
3. Le niveau de données et le référentiel applicatif
4. L'intégration des modules fonctionnels
5. Transactions, verrous et performances
6. Évolution, migration et architectures de remplacement

Chapitre 1.1

Le modèle trois niveaux

Trois responsabilités, trois niveaux remplaçables. Toute la valeur du modèle tient dans cette phrase.

Le modèle trois niveaux, aussi appelé architecture à trois niveaux ou architecture de présentation, sépare l'application en une couche de présentation, une couche de traitement et une couche de stockage. L'origine du découpage est simple : les trois évoluent à des rythmes différents. Un poste de travail se remplace tous les trois ans, un serveur tous les cinq, une base tous les huit. Les faire évoluer ensemble, c'est remplacer les trois pour le plus jeune d'entre eux.

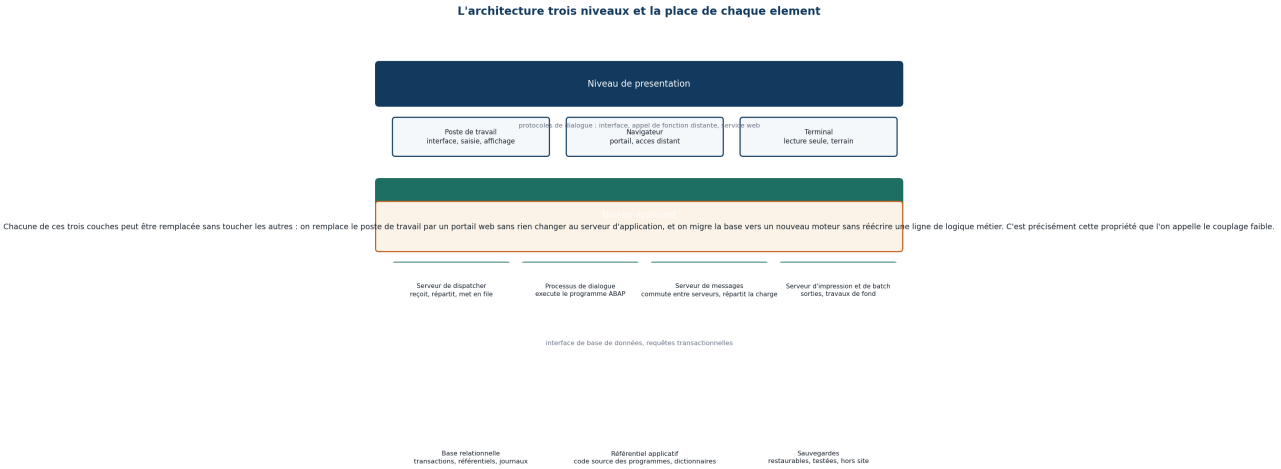


Figure 1 – L'architecture trois niveaux et la place de chaque élément

Niveau	Responsabilité	Composants typiquesProtocole vers le niveau suivant	
Présentationcapter l'action, afficher le résultat, valider la saisie	client lourd, navigateur, portail, terminal	protocole de dialogue, appel de fonction distante, service web	
Traitement	appliquer les règles de gestion, contrôler les droits, orchestrer la base	serveur de répartition, processus de traitement, serveur de messages	interface de base de données, instruction de contrôle
Données	conserver l'état, garantir l'intégrité, servir les lectures	moteur relationnel, stockage, sauvegardes	interface native du moteur, procédure stockée

Tableau 1 – Les trois niveaux, leurs responsabilités et leurs protocoles

Le protocole est la vraie frontière. Parce qu'il est normalisé, il permet de remplacer un niveau sans toucher les deux autres – c'est la définition opérationnelle du couplage faible.

Definition – Couplage faible

Deux composants sont faiblement couplés lorsque l'un peut être remplacé sans que l'autre ait à être modifié. Dans une architecture trois niveaux, le couplage faible se mesure à la frontière standardisée normalisée qui sépare chaque couche. Un protocole documenté, stable et indépendant du produit suffit ; c'est la raison pour laquelle l'appel de fonction distante a remplacé durablement les accès directs au moteur de base de données.

Astuce

Pour vérifier le couplage faible d'un système, posez une seule question : si l'on remplace entièrement le niveau de données, quelle part du code faut-il réécrire ? Si la réponse est « la totalité », il n'y avait pas d'architecture trois niveaux, il n'y avait qu'une application avec des couches dessinées.

A retenir

- Le découpage existe parce que les trois niveaux n'évoluent pas au même rythme.
- Le protocole normalisé entre les niveaux est la frontière qui rend le remplacement possible.
- Un système fortement couplé n'a pas trois niveaux : il a trois couches sur une seule application.

Chapitre 1.2

Le niveau applicatif : serveur, répartition et interpréteur

Le cœur du système d'entreprise n'est pas la base de données : c'est le processus qui sait quoi en faire.

Le niveau de traitement est le plus riche et le plus difficile à remplacer. Il porte la logique métier, les contrôles d'accès, les règles de gestion, et l'orchestration des accès à la base. Dans les systèmes d'entreprise de troisième génération, ce niveau est constitué de quelques composants précis, dont il faut comprendre le rôle exact pour raisonner sur les performances.

- **Serveur de répartition :** point d'entrée unique. Il reçoit la requête, vérifie l'authentification, choisit un processus de traitement libre, lui transmet le contexte utilisateur, puis ne fait plus qu'attendre la réponse. Il ne connaît aucune règle métier.
- **Processus de traitement :** l'exécutant. Il interprète le programme, dialogue avec la base, tient l'état de la transaction. Il y en a plusieurs par serveur, et c'est leur nombre qui borne la charge simultanée.
- **Serveur de messages :** le commutateur. Il centralise les communications entre serveurs, permet l'équilibrage de charge, distribue les données entre serveurs, et sert de mémoire partagée pour les tables de configuration.
- **Serveur de travaux de fond :** exécute ce qui ne doit pas bloquer l'utilisateur :les mises à jour massives, exports, recalculs, nettoyages.
- **Serveur d'impression et de sortie :** produit les documents, rend les lots de données pour des traitements externes.

Le dimensionnement du nombre de processus

Le nombre de processus de traitement par serveur est le principal levier de performance d'un système d'entreprise, et le plus souvent mal réglé. Trop peu, la file d'attente s'allonge et les utilisateurs attendent. Trop, les processus se disputent la mémoire et le système d'exploitation commence à échanger avec le disque, ce qui dégrade tout le reste. La valeur correcte dépend de la taille du contexte mémoire par processus et de la mémoire disponible — aucune valeur par défaut n'est universelle.

Transaction coûteuse	Pourquoi elle coûte cherParade recommandée	
Écran long affichant une liste de milliers de lignesla liste est construite en mémoire, la transaction reste ouverte	limiter le nombre de lignes, imposer un filtre, paginer	
Modification massive en boucle dans un seul écran	une seule transaction enorme, verrouillage prolongé	fractionner en paquets et valider entre chaque
Appel synchrone vers un autre système pendant la transaction	le temps d'attente externe bloque les ressources de traitement	mettre l'appel en file de sortie, traiter après validation
Sélection sans critère d'index	lecture complète de la table dans la mémoire du serveur de base	indexer le critère de filtre, ou traiter en différé
Verrouillage excessif sur une table très sollicitée	les transactions concurrentes se serialisent	regrouper les mises à jour, réduire la granularité des verrous

Tableau 1 – Les quatre transactions les plus coûteuses et leur parade

Les quatre premières causes représentent la très grande majorité des problèmes de performance de ce type de système. Elles relèvent toutes de la discipline de conception, pas de l purchases de matériel supplémentaire.

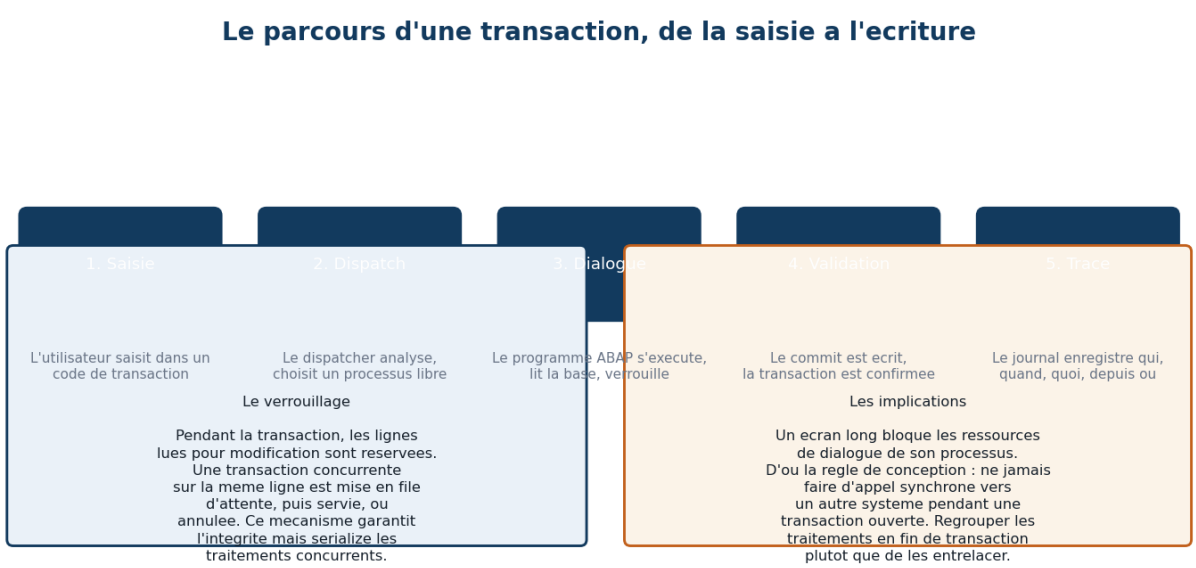


Figure 1 – Le parcours d'une transaction, de la saisie à l'écriture

Algorithme 1 – Traitement d'une requête utilisateur dans l'architecture trois niveaux

```
01  Le poste de travail envoie la requête au serveur de répartitionLe serveur de répartition
authentifie l'utilisateur et ouvre un contexte
02  Il sélectionne un processus de traitement libre dans la file
03  Il lui transmet le contexte, puis attend la réponse
04  Le processus de traitement charge l'écran en exécutant le programme associé
05  Le programme émet des instructions vers la base de données
06  La base lit et verrouille les lignes concernées, puis rend le résultat
07  Le programme effectue les contrôles de gestion et les contrôles d'accès
08  Le processus valide la transaction : la base écrit définitivement
09  Le résultat est renvoyé au serveur de répartition, puis au poste de travail
```

Entrées : Identifiant utilisateur, code de transaction, données saisies • **Sorties :** Écran affiché, écritures validées, entrées de journal • **Complexité :** Latence = aller-retour réseau + temps de traitement + temps d'accès base

Les étapes 3 et 9 sont les deux points de contention. La troisième est un goulot d'étranglement par conception ; la neuvième est la seule étape qui rend le résultat durable, et celle dont l'exécution conditionné toutes les autres.

A retenir

- Le serveur de répartition ne connaît aucune règle métier : son rôle est d'aiguiller et de contrôler l'accès.
- Le nombre de processus de traitement est le premier levier de performance et le plus souvent mal réglé.
- Un appel synchrone vers l'extérieur pendant une transaction ouverte bloque des ressources pendant toute l'attente.
- Un écran qui liste des milliers de lignes est un problème de conception, pas de capacité.

Chapitre 1.3

Le niveau de données et le référentiel applicatif

Deux dépôts de données coexistent : celui que la base contient, et celui que le code applicatif définit.

Le niveau de données d'un système d'entreprise ne se résume pas à un moteur relationnel et ses tables. Il comprend un second dépôt, tout aussi déterminant, qui contient la définition même des programmes : ce que la base contient est l'état du système, ce que le référentiel contient est le système lui-même.

	Base de données	Référentiel applicatif
Contient	les transactions, les données de référence, les journauxles programmes, les dictionnaires, les définitions d'écrans, les variantes de configuration	
Modifié par	les transactions métier	les mises à jour de version et les extensions
Restauration	sauvegarde et restauration de base	mise à jour de version avec reprise automatique
Rôle	conserver l'état	définir le comportement
Conséquence d'une perte	reprise à partir d'une sauvegarde	retour à la version précédente ou réinstallation
Sauvegarde	quotidienne, incrémentale, testée	sauvegarde avant mise à jour, avec reprise

Tableau 1 – Base de données et référentiel applicatif

La distinction n'est pas académique : une mise à jour de version qui ne préserve pas le contenu étendu ne met pas à jour le système, elle le détruit. C'est la raison pour laquelle toute migration commence par un inventaire écrit du référentiel.

$$\mathrm{Coût}(S, T) = \underbrace{\mathrm{Acc\text{-}base}(S, T)}_{\square 0 \square} + \underbrace{\mathrm{CRUD}(S, T)}_{\square 0 \square} + \underbrace{\mathrm{Join}(S, T)}_{\text{relations entre tables}} \quad \text{(E.1)}$$

La partition en trois termes n'est pas un détail technique : elle est la trame d'analyse de toute optimisation. Réduire le premier terme se fait par des index, le deuxième par l'architecture de la logique métier, le troisième par la normalisation – et les trois entrée en conflit.

- **Normalisation :** réduire la redondance pour garantir la cohérence, au prix d'un nombre de jointures plus élevé. L'optimum se trouve rarement à la forme normale la plus stricte.
- **Dé-normalisation :** dupliquer délibérément pour speeding la lecture, en assumant qu'un contrôle applicatif maintient la cohérence. Valable pour une tablerequently lue, fausse dès qu'elle s'écrit.
- **Index :** accélère la sélection, ralentit l'écriture et occupe de l'espace. Il doit correspondre à un critère de filtre réel, sinon il ne sert qu'à ralentir.

- **Tâches de fond :** reconstruire les index, mettre à jour les statistiques, vérifier la cohérence. Elles déverrouillent et dégradent les performances pendant leur exécution : il faut les planifier, pas les découvrir.

Auditer un niveau de données en huit points

- **Volume** de chaque table, en lignes et en octets, et période de croissance.
2. **Index** présents, et pour chacun, le critère de filtre auquel il correspond réellement.
3. **Redondances** assumées, et contrôle applicatif qui les maintient.
4. **Jointures** les plus fréquentes, et nombre d'enregistrements ramenés.
5. **Tables sans mouvement** depuis un an, à archiver.
6. **Tâches de fond** en place, et leur impact sur la charge diurne.
7. **Sauvegarde** : fréquence, durée, et date du dernier test de restauration réel.
8. **Inventaire du référentiel** : extensions, zs, programmes personnalisés, et propriétaire de chacun.

Une sauvegarde dont la restauration n'a jamais été pratiquée n'est pas une sauvegarde. Le test consiste à restaurer une copie sur un environnement distinct, à vérifier le compte rendu de restauration, puis à ouvrir un échantillon de transactions et à les rapprocher des pièces comptables correspondantes. La durée de conservation de la sauvegarde doit couvrir le délai de détection d'un incident, pas seulement le délai de reprise : c'est souvent ce second délai qui est sous-estimé.

La sauvegarde jamais testée

Astuce

La date du dernier test de restauration est l'indicateur de fiabilité le plus honnête d'un système de données. Tout le reste — procédure, documentation, contrat de sauvegarde — décrit une intention.

A retenir

- Le référentiel contient le système ; la base n'en contient que l'état instantané.
- Toute mise à jour commence par un inventaire écrit du contenu étendu.
- L'optimisation d'une base se raisonne en trois termes : cache, écritures, jointures.
- La seule mesure honnête de fiabilité d'une sauvegarde est la date du dernier test de restauration réel.

Chapitre 1.4

L'intégration des modules fonctionnels

L'avantage d'un progiciel intégré n'est pas la richesse des fonctions : c'est l'absence de ressaisie.

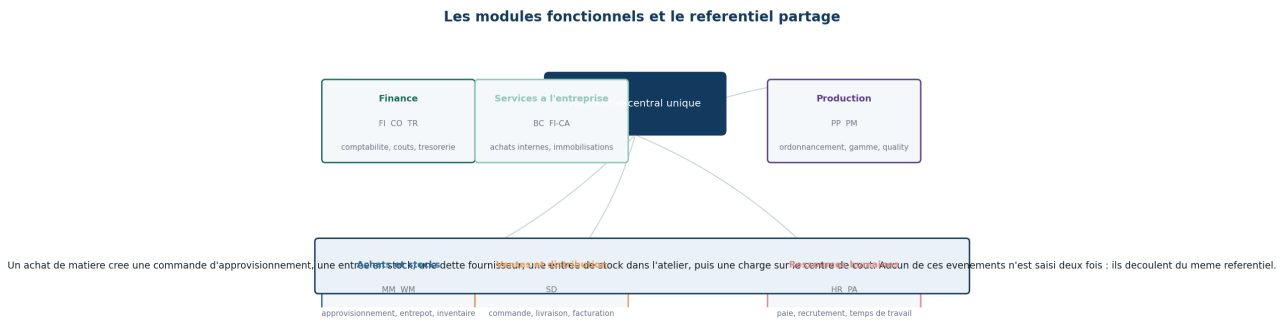


Figure 1 – Les modules fonctionnels et le référentiel partagé

Un progiciel intégré juxtapose des modules fonctionnels — finance, achats, stocks, ventes, production, ressources humaines — qui partagent le même référentiel et la même base. La conséquence pratique est considérable : un même fait élémentaire n'est jamais saisi deux fois, et sa propagation dans tous les modules concernés est automatique.

Algorithme 2 – Traversée d'un achat de matière à travers les modules

```
Un besoin d'achat est exprimé par le module de productionLe module des achats transforme le besoin en commande fournisseur
À réception, le module de stocks enregistre l'entrée en magasin
Le module financier crée automatiquement la dette envers le fournisseur
La matière est consommée par un ordre de fabrication
Le module de production valorise la consommation sur le centre de coût
Le module financier impute cette valeur sur le compte de résultat concerné
Le module de contrôle de gestion restitue l'écart entre prévision et réalisation
```

Entrées : Besoin d'achat, quantité, centre de coût, date • **Sorties :** Commande, entrée en stock, écritures, écart analysable • **Complexité :** Huit modules touchés, une seule saisie utilisateur

C'est le trajet type d'un système intégré. Aucune des étapes intermédiaires n'est saisie : chacune dérive de la précédente. C'est là que réside l'économie, et c'est aussi là que réside le risque : une erreur en amont se propage partout sans qu'aucune revalidation ne l'arrête.

Module	Domaine	Entrées principales	Sorties principales
Finance	comptabilité générale, contrôle de gestion, trésorerieécritures d'exploitation, pièces justificatives	bilan, compte de résultat, plan de trésorerie	
Achats et approvisionnement	fournisseurs, commandes, réception	besoins, contrats	commandes, factures, réception
Stocks	mouvements, inventaires, valorisations	réceptions, sorties, transferts	état du stock, valorisation
Ventes et distribution	clients, commandes, livraison, facturation	commandes, disponibilité	bons de livraison, factures
Production	ordonnancement, gamme, suivi de fabrication	noms de production, besoins	coûts, quantités produites
Ressources humaines	paie, temps de travail, recrutement	pointages, contrats	bulletins, provisions sociales

Tableau 1 – Les six modules d'un progiciel intégré et leurs flux

Definition – Ligne analytique

Le mécanisme qui rattache une écriture à un axe d'analyse : centre de coût, centre de profit, projet, article. C'est lui qui permet de répondre à « combien cette activité a-t-elle coûté » sans dupliquer les comptes. Sans ligne analytique, le module de contrôle de gestion n'a rien à analyser, et l'intégration perd sa contrepartie : on a gagné la suppression de la ressaisie en perdant l'analyse.

Le risque de l'intégration excessive

Chaque module qui se verrouille sur une donnée d'un autre module crée une dépendance. Un jour, la refonte d'un module impose de revenir dans les six autres. La parade n'est pas de proscrire l'intégration — c'est ce qui a été acquis — mais de fixer une règle : les modules ne s'appellent jamais directement entre eux. Une donnée qui doit traverser deux modules transite par un interface documentée, ce qui rend le remplacement d'un module possible sans modifier les autres.

Séparation des responsabilités et piste d'audit

Dans un système intégré, l'accès à une fonction sensible est régulé par un profil et non par une habilitation de masse. La séparation des responsabilités impose qu'aucun acteur ne puisse seul créer puis valider une écriture critique. La piste d'audit doit permettre de reconstituer qui a fait quoi, depuis quel poste, à quelle date, avec le document à l'appui — y compris lorsque l'écriture a été générée automatiquement par un autre module, cas où l'automaticité est précisément ce qu'il faut savoir tracer.

A retenir

- L'intégration supprime la ressaisie, mais propage aussi les erreurs sans point de contrôle intermédiaire.
- Les modules ne doivent jamais s'appeler directement : ils échangent par une interface documentée.
- La ligne analytique est ce qui rend l'intégration utile à l'analyse, et non seulement à la saisie.
- La séparation des responsabilités impose qu'aucun acteur ne puisse créer puis valider seul une écriture critique.

Chapitre 1.5

Transactions, verrous et performances

Le coût d'une transaction ne dépend pas de ce qu'elle calcule, mais de la durée pendant laquelle elle retient les lignes.

Le moteur relationnel garantit qu'une suite d'écritures est soit entièrement appliquée, soit entièrement annulée. Ce garantie a un prix : les lignes lues pour modification sont réservées jusqu'à la validation, et une transaction concurrente qui touche la même ligne est mise en attente. La performance d'un système d'entreprise se joue donc moins dans le calcul que dans la durée de réservation.

$$\text{Réservation}(l) = t_{\square 1 \square} - t_{\square 2 \square}$$
$$\text{Attente}(n) \leq \frac{D}{n} \quad \text{où } D \text{ est la $$durée moyenne d'une transaction concurrente sur la même ligne} \tag{E.2}$$

La première expression est la seule qui compte en pratique. Halver la durée de réservation d'une transaction équivaut à doubler le débit, sans toucher au matériel. La seconde explique pourquoi les points de contention deviennent des goulots d'étranglement bien avant la saturation des ressources.

Niveau	Levier	Gain attendu	Coût de mise en œuvre
Présentation	limiter le nombre de lignes renvoyéesfort, immédiat	faible, conception d'écran	
Traitement	réserver moins de lignes par transaction	fort, immédiat	faible, discipline de programmation
Traitement	augmenter le nombre de processus	moyen, plafonné	moyen, dimensionnement matériel
Données	ajouter un index sur un critère réel	fort, immédiat	faible, mais maintenance
Données	dé-normaliser une table très lue	fort en lecture	moyen, contrôle applicatif à écrire
Données	réduire le nombre de tables jointes	moyen	élevé, refonte du modèle
Matériel	plus de mémoire pour le serveur de base	moyen, souvent fort	élevé
Matériel	dupliquer le serveur applicatif	faible si la contention est dans les données	élevé, sans benefit si la cause est elsewhere

Tableau 1 – Leviers de performance par niveau, et leur rendement

Les quatre premières lignes sont gratuites ou peu coûteuses, et règlent plus de 80 % des cas. Acheter du matériel avant d'avoir examiné la durée de réservation des transactions est l'erreur la plus fréquente et la plus coûteuse.

Si deux transactions se disputent les mêmes lignes, doubler le nombre de serveurs d'application ne fait qu'ajouter des clients à la même file d'attente. Le temps d'attente augmente, le débit ne bouge pas, et l'on a payé deux fois. Le diagnostic est simple : si le temps d'attente des transactions dépasse le temps de leur exécution, le goulot est dans les données, pas dans le nombre de serveurs.

Multiplier les serveurs applicatifs pour résoudre un problème de données

Travaux pratiques 1.1 — Diagnostiquer un ralentissement jusqu'à sa cause

Duree : 3 h | Niveau : Avance | Outils : Un jeu de transactions avec horodatages de début, de fin et de validation, et une table de contention par ressource

Analyser un système d'entreprise ralenti, distinguer goulot de données et manque de puissance, et produire un diagnostic argumenté.

Deroulement

1. Calculer la durée moyenne de réservation de chaque transaction, et le temps d'attente moyen associé.
2. Distinguer les transactions dont l'attente dépasse la durée d'exécution : elles subissent un conflit.
3. Regrouper les transactions en conflit par ressource, et identifier les trois tables les plus disputées.
4. Proposer pour chacune des trois tables une réduction de la portée de la réservation, et justifier.
5. Comparer le gain théorique attendu à une augmentation de 50 % du nombre de serveurs applicatifs, et conclure.
6. Écrire en trois lignes ce qu'il faut faire en premier, et pourquoi.

Livrable attendu

- T
- a
- b
- l
- e
- a
- u
- d
- e
- s
- d
- u
- r
- é
- e
- s
- d
- e
- r
- é
- s
- e
- r
- v

- a
- t
- i
- o
- n
- e
- t
- d
- '
- a
- t
- t
- e
- n
- t
- e
- ,
- c
- l
- a
- s
- s
- e
- m
- e
- n
- t
- d
- e
- s
- c
- o
- n
- f
- l
- i
- t
- s
- ,

- n
- o
- t
- e
- d
- '
- a
- n
- a
- l
- y
- s
- e
- d
- e
- t
- r
- o
- i
- s
- l
- i
- g
- n
- e
- s
- .

Bareme indicatif

Critere	Points
La note conclut que la contention est dans les données4	
Le gain annoncé est justifié par le calcul	4
La comparaison avec l'ajout de serveurs est chiffrée	2

A retenir

- Le coût d'une transaction tient à la durée de sa réservation, pas à la complexité de son calcul.
- Halver la durée de réservation double le débit, sans matériel supplémentaire.
- Si le temps d'attente dépasse le temps d'exécution, le goulot est dans les données.
- Le premier réflexe de performance est l'examen de la durée de réservation, jamais l'achat de serveurs.

Chapitre 1.6

Évolution, migration et architectures de remplacement

La question n'est pas de migrer vers une plateforme nouvelle, mais de rendre le modèle métier indépendant de la plateforme.

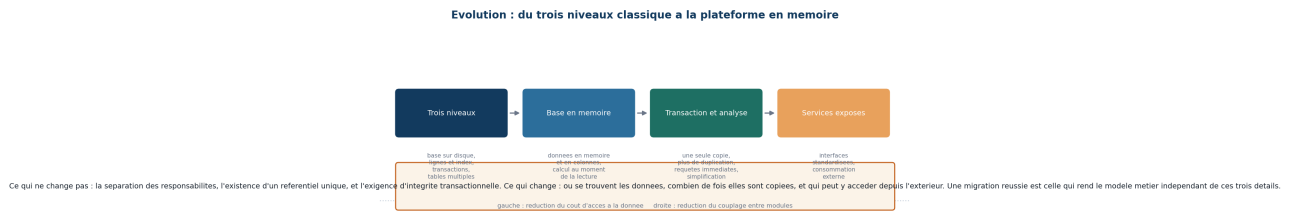


Figure 1 – Du trois niveaux classique à la plateforme en mémoire

Architecture	Unité de déploiement	Base de donnéesCe qui la caractérise	
Trois niveaux	un serveur, une baserelationnelle sur disque,à Accès	simplplicité conceptuelle, forge de compétences	
Services distribués	plusieurs services indépendants	une base par service	déploiement indépendant, pannes partielles, complexité d'exploitation
Plateforme en mémoire	un socle unique	unique, en colonnes, calcul au moment de la lecture	rapidité, simplification du modèle, migration coûteuse

Tableau 1 – Les trois architectures et ce qui les distingue

Le passage d'une architecture à l'autre ne change pas la question de fond : où se trouve la logique métier, et combien de fois la donnée est-elle dupliquée.

- **Base en mémoire :** les données tiennent en mémoire, en colonnes, avec compression. L'accès ne fait plus d'aller-retour disque, et le calcul analytique s'exécute au moment de la lecture plutôt que dans un entrepôt séparé.
- **Transaction et analyse réunies :** une seule copie de la donnée, au lieu de trois — base transactionnelle, entrepôt analytique, système de données. La duplication disparaît, et avec elle une grande partie des problème de cohérence.
- **Services standardisés :** les fonctions deviennent des interfaces que d'autres systèmes consomment, ce qui crée une dépendance applicable dans l'autre sens : le fournisseur de l'interface devient un point de contrôle.

Migrer sans casser : la séquence qui fonctionne

- **Inventaire écrit du référentiel** : extensions, personnalisations, programmes non standard, avec un propriétaire pour chacun. Sans cet inventaire, la migration est une devinette.

2. **Conception du modèle cible** en termes fonctionnels, et non en tables. C'est le seul niveau où le métier doit décider.
3. **Gel des développements** sur l'ancien système pendant toute la migration, et report des besoins sur le nouveau.
4. **Migration des données par lots répétés**, avec un contrôle de rapprochement à chaque itération, jamais une seule fois à la fin.

5. **Reprise des.personnalisations une par une**, en justifying pour chacune :utile, ou remodelable par une configuration.
6. **Bascule par domaine fonctionnel**, jamais tout d'un coup, avec une période de double exploitation.
7. **Plan de retour arrière** écrit et testé avant la bascule, avec le délai maximal de retour decided à l'avance.

Ce qu'une migration ne supprime pas

Une plateforme en mémoire rend la requête rapide, pas l'analyse juste. Si les règles analytiques sont écrites à la main dans des requêtes, elles migrent telles quelles, avec leurs erreurs. De même, la réactivité d'une requête ne rend pas l'utilisateur rapide : si l'écran liste mille lignes, l'utilisateur attend autant. La migration améliore les fondations ; elle ne dispensé d'aucun travail de conception, et il faut annoncer cela avant de commencer.

Astuce

La meilleure préparation à une migration est de rendre le modèle métier indépendant du détail technique. Un modèle bien écrit, exprimé en objets métier et non en tables, réduit le coût de toute migration future — y compris celle qu'on ne peut pas encore prévoir. C'est un investissement dont le rendement est différé mais certain.

Projet – Rédiger un dossier d'architecture trois niveaux

Duree : 6 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Une PME manufacturière souhaité remplacer son progiciel de gestion par un système en architecture trois niveaux, initialement sur le modèle d'entreprise, avec un objectif de migration à moyen terme vers une plateforme en mémoire.

Missions

1. Décrire les trois niveaux, leurs composants et le protocole qui relié chacun au suivant, en justifiant chaque choix par un critère d'exploitation.
2. Dimensionner le nombre de processus de traitement et l'architecture du serveur, en explicitant les grandeurs retenues.
3. Établir le plan de sauvegarde des deux dépôts distincts — base et référentiel — avec la fréquence, la durée de conservation et la procédure de test.
4. Rédiger le plan de reprise après panne, incluant le délai de détection, le délai de restauration et le seuil au-delà duquel les écritures sont basculées dans une file de reprise.
5. Établir l'inventaire du contenu étendu du référentiel et la procédure de qualification de chaque personnalisation.
6. Chiffrer la séquence de migration en six étapes, avec pour chacune sa condition d'entrée, sa sortie et son critère de retour arrière.

Livrables

- D
- o
- c
- u
- m
- e
- n
- t

- d
- '
- a
- r
- c
- h
- i
- t
- e
- c
- t
- u
- r
- e
- e
- n
- t
- r
- o
- i
- s
- n
- i
- v
- e
- a
- u
- x
- ,
- p
- l
- a
- n
- d
- e
- s
- a
- u
- v

- e
- g
- a
- r
- d
- e
- e
- t
- d
- e
- r
- e
- p
- r
- i
- s
- e
- ,
- i
- n
- v
- e
- n
- t
- a
- i
- r
- e
- d
- u
- r
- é
- f
- é
- r
- e
- n
- t
- i

- e
- l
- ,
- s
- é
- q
- u
- e
- n
- t
- i
- e
- l
- d
- e
- m
- i
- g
- r
- a
- t
- i
- o
- n
- ,
- e
- t
- t
- a
- b
- l
- e
- a
- u
- d
- '
- e
- s
- t

- i
- m
- a
- t
- i
- o
- n
- d
- e
- l
- a
- c
- h
- a
- r
- g
- e
- p
- a
- r
- é
- t
- a
- p
- e
- .

Évaluation

- L
- e
- j
- u
- r
- y
- v
- é
- r
- i
- f

- i
- e
- q
- u
- e
- c
- h
- a
- q
- u
- e
- d
- é
- c
- i
- s
- i
- o
- n
- d
- e
- d
- i
- m
- e
- n
- s
- i
- o
- n
- n
- e
- m
- e
- n
- t
- c
- i
- t

- e
- l
- a
- g
- r
- a
- n
- d
- e
- u
- r
- m
- e
- s
- u
- r
- é
- e
- q
- u
- i
- l
- a
- j
- u
- s
- t
- i
- f
- i
- e
- ,
- q
- u
- e
- l
- a
- r

- e
- p
- r
- i
- s
- e
- a
- p
- r
- è
- s
- p
- a
- n
- n
- e
- c
- o
- u
- v
- r
- e
- l
- e
- c
- a
- s
- d
- e
- l
- a
- p
- e
- r
- t
- e
- d
- u

- r
- é
- f
- é
- r
- e
- n
- t
- i
- e
- l
- ,
- e
- t
- q
- u
- e
- l
- a
- m
- i
- g
- r
- a
- t
- i
- o
- n
- p
- r
- é
- v
- o
- i
- t
- u
- n
- r

- e
- t
- o
- u
- r
- a
- r
- r
- i
- è
- r
- e
- t
- e
- s
- t
- é
- .

A retenir

- L'architecture trois niveaux reste la grille de lecture de tout système d'entreprise, quel que soit son âge.
- Ce qui change entre les générations, c'est la localisation de la donnée et le nombre de ses copies, pas le découpage des responsabilités.
- Une migration se prépare par un inventaire écrit du référentiel, pas par un choix de plateforme.
- Le retour arrière doit être dimensionné, testé et daté avant la bascule, pas improvisé après l'incident.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
API	Application Programming Interface	Interface d'échange entre deux logiciels.
CRUD	Create, Read, Update, Delete	Quatre opérations de base sur un enregistrement.
EDI	Electronic Data Interchange	Échange de messages normalisés entre systèmes d'information.
ERP	Enterprise Resource Planning	Système d'information couvrant l'ensemble des fonctions de l'entreprise.
FO	Flux d'ordre	Circuit d'approvisionnement, du besoin à la réception.
ICI	Interconnexion des systèmes d'information	Ensemble des liaisons entre applications.
OLTP	Online Transaction Processing	Traitement transactionnel en ligne.
PME	Petite et moyenne entreprise	Entreprise de moins de 250 salariés.
RGPD	Règlement général sur la protection des données	Cadre européen de protection des données personnelles.
SAV	Service après-vente	Fonction de traitement des retours clients.
SCM	Supply Chain Management	Gestion de la chaîne d'approvisionnement.
WMS	Warehouse Management System	Logiciel de gestion d'entrepôt.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées E, les lois de données D, les espaces de paramètres Θ.

Symbole	Signification
N	Nombre de sites reliés.
E	Nombre de flux échangés entre deux sites.
G	Graphe des interdépendances : les sites et les flux.
$u \in V(G)$	Site du graphe.
c_{ij}	Coût du flux qui va de i vers j.
T_{ij}	Temps de traversée du flux qui va de i vers j.
S_i	Stock disponible au site i.
D_j	Demande adressée au site j.
δ	Taux de service, part des demandes servies dans le délai.
k	Nombre de niveaux de la cascade de décision.
p_i	Priorité du critère i.
w_i	Pondération du critère i dans une note composite.

Tableau 1 – Symboles mathématiques et leur signification

Convention

Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

A retenir

- Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
- Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
- Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. L'architecture trois niveaux et la place de chaque élément
1. Le parcours d'une transaction, de la saisie à l'écriture
1. Les modules fonctionnels et le référentiel partagé
1. Du trois niveaux classique à la plateforme en mémoire

Tableaux

1. Les trois niveaux, leurs responsabilités et leurs protocoles
1. Les quatre transactions les plus coûteuses et leur parade
1. Base de données et référentiel applicatif
1. Les six modules d'un progiciel intégré et leurs flux
1. Leviers de performance par niveau, et leur rendement
1. Les trois architectures et ce qui les distingue
1. Abréviations employées dans le cours

1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.